

kurs języka C++

permutowanie i sortowanie tablic

Instytut Informatyki
Uniwersytetu Wrocławskiego

Paweł Rzechonek

Prolog

Losowa permutacja to losowe rozmieszczenie elementów zadanego ciągu, przy czym każde ustawienie jest tak samo prawdopodobne. Dla n -elementowego ciągu istnieje $n!$ różnych ustawień. Najefektywniejszym sposobem generowania losowej permutacji jest algorytm Fishera-Yatesa (nazywany też algorytmem Knutha), który w liniowym czasie $O(n)$ zamienia elementy miejscami, gwarantując jednostajny rozkład (każde ustawienie jest tak samo prawdopodobne).

Sortowanie to przeciwieństwo mieszania (losowa permutacja) i polega na uporządkowaniu ciągu danych według określonego kryterium – rosnąco, malejąco, leksykograficznie itp. Jest to fundamentalne zagadnienie w informatyce, ułatwiające potem efektywne przeszukiwanie i analizę danych. Podstawowe algorytmy sortujące to sortowanie bąbelkowe, przez wstawianie, przez wybieranie, sortowanie szybkie (przez podział) oraz sortowanie przez scalanie.

Zadanie

Najpierw zdefiniuj szablon funkcji drukującej dane zgromadzone w tablicy. Domyślnie drukuj na standardowym wyjściu `std::cout` (ostatni domyślny argument).

```
template<typename T>
void arrprint(const T arr[], int siz, ostream &out = cout) {
    ...
}
```

Potem zdefiniuj szablon funkcji kopiującej dane z jednej tablicy do drugiej.

```
template<typename T>
void arrcopy(const T from[], int siz, T to[]) {
    ...
}
```

Następnie zdefiniuj szablon funkcji dokonującej losowej permutacji danych umieszczonych w tablicy – zaprogramuj algorytm Fishera-Yatesa. W algorytmie tym musimy skorzystać z jakiegoś generatora liczb pseudolosowych – można założyć, że nawet najprostszy generator dostępny w C++ generuje liczby z rozkładem jednostajnym. Wykonaj po kilka niezależnych testów dla danych różnego typu:

- liczby `int` lub `double`;
- napisy `string`;
- ciągi znakowe `const char*`;
- jakaś własna prosta klasa/struktura, np. `osoba`, `kolor`, `wymierna` itp.

Permutacje testuj na danych skopiowanych z tablicy z wartościami wzorcowymi.

```
template<typename T>
void arrpermut(T arr[], int siz) {
    ...
}
```

Wreszcie na koniec zdefiniuj co najmniej dwa szablony funkcji sortujących dane znajdujące się w tablicy, wykorzystujące w procesie sortowania tylko porównania i zamiany elementów. Jedna z funkcji niech będzie implementacją jakiejś prostej metody opartej na zamianach sąsiednich elementów (sortowanie bąbelkowe albo przez wstawianie) natomiast druga niech działa w oparciu o algorytm typu „dziel i zwyciężaj” (sortowanie szybkie albo przez scalanie).

```
template<typename T, typename Comp>
void arrsort(T arr[], int siz) {
    ...
}
```

Szablony tych funkcji powinny mieć dodatkowy parametr określający strategię porównywania elementów (domyślnie niech będzie to strategia bazująca na operatorze `<`) – taki trejt może być obiektem funkcyjnym, w którym operator wywołania funkcji z dwoma argumentami tego samego typu co elementy tablicy zwraca wartość boolowską (wynik porównania). Zdefiniuj dwa takie trejty: jeden do sortowania w kolejności rosnącej/niemalejącej a drugi do sortowania kolejności malejącej/nierosnącej.

```
template<typename T>
class less {
    ...
    bool operator()(const T &a, const T &b);
    ...
}
```

Szablony wszystkich funkcji umieść w osobnym pliku nagłówkowym.

Ważne elementy programu

- Definicja szablonów funkcji do drukowania, kopiowania i permutowania tablic.
- Definicja szablonów funkcji do sortowania tablic.
- Definicja trejtów realizujących porównania.
- Realizacja specjalizacji dla napisów typu `const char*`.
- Podział programu na pliki nagłówkowe i pliki źródłowe (wyodrębniony osobny plik z funkcją `main()` z testami).